

BUILD WEEK 3 // ESERCIZIO 2

Analisi Server Linux e Servizi TCP

Gestione processi, mappatura porte di rete
e banner grabbing con telnet

CyberOps Workstation VM | Ambiente Controllato

AUTORE	Leandro Ancona
CORSO	Cyber Security & Ethical Hacking -- EPICODE
DATA	22 giugno 2026
AMBIENTE	Macchina Virtuale CyberOps Workstation
STRUMENTI	ps, netstat, telnet
PROTOCOLLI	TCP (HTTP, SSH, FTP), UDP (DHCP)



00

Indice

01 Obiettivi del Laboratorio

02 Parte 1 -- Gestione e Analisi dei Processi

- 1.1 Identificazione dei Processi Attivi con ps
- 1.2 Gerarchia dei Processi con ps -ejH
- 1.3 Mappatura delle Porte con netstat -tunap
- 1.4 Analisi dell'Architettura di Sicurezza di Nginx

03 Parte 2 -- Validazione dei Servizi TCP (Banner Grabbing)

- 2.1 Test del Web Server -- Porta 80 (HTTP)
- 2.2 Test del Server SSH -- Porta 22
- 2.3 Test della Porta UDP 68 -- Fallimento Atteso

04 Conclusioni e Sintesi Comparativa

01

Obiettivi del Laboratorio

Questo laboratorio ha l'obiettivo di sviluppare competenze pratiche nell'analisi dei processi di sistema e dei servizi di rete su un host Linux. Le attività si articolano in due macro-aree complementari: la gestione e l'ispezione dei processi tramite utility native, e la validazione diretta dei servizi TCP attraverso tecniche di **banner grabbing**.

Strumenti utilizzati	ps, netstat, telnet
Protocolli analizzati	TCP (HTTP, SSH, FTP), UDP (DHCP)
Ambiente	CyberOps Workstation -- VM Linux
Data	22 Giugno 2026

Ogni comando eseguito e' documentato con uno screenshot reale della sessione di lavoro. Le osservazioni critiche sono evidenziate in riquadri dedicati.

02

Parte 1 -- Gestione e Analisi dei Processi

- 1.1 -- Identificazione dei Processi Attivi con ps

Il primo passo e' stato eseguire il comando base **ps** senza argomenti. Questo comando, per design, restituisce esclusivamente i processi associati alla sessione terminale corrente (shell bash e il comando stesso). Il risultato e' deliberatamente limitato: il kernel espone solo cio' che appartiene all'utente corrente nella sessione attiva.

Fig. 1 -- Output di ps senza argomenti: visibilita' limitata a bash e ps (PID 1278, 1282)

```
analyst@secOps ~]$ sudo ps -ejH
  PID   PGID   SID TTY          TIME CMD
    2     0     0 ?        00:00:00 kthreadd
    3     0     0 ?        00:00:00 pool_workqueue_release
    4     0     0 ?        00:00:00 kworker/R-rcu_gp
    5     0     0 ?        00:00:00 kworker/R-synct_wq
    6     0     0 ?        00:00:00 kworker/R-kvfree_rcu_reclaim
    7     0     0 ?        00:00:00 kworker/R-slab_flushwq
    8     0     0 ?        00:00:00 kworker/R-netns
   12     0     0 ?        00:00:00 kworker/u8:0-events_unbound
   13     0     0 ?        00:00:00 kworker/u8:1-ipv6_addrconf
   14     0     0 ?        00:00:00 kworker/R-mm_percpu_wq
   15     0     0 ?        00:00:00 ksoftirqd/0
   16     0     0 ?        00:00:02 rcu_preempt
   17     0     0 ?        00:00:00 rcub/0
   18     0     0 ?        00:00:00 rcu_exp_par_gp_kthread_worker/0
   19     0     0 ?        00:00:00 rcu_exp_gp_kthread_worker
   20     0     0 ?        00:00:00 migration/0
   21     0     0 ?        00:00:00 idle_inject/0
   22     0     0 ?        00:00:00 cpuhp/0
   23     0     0 ?        00:00:00 cpuhp/1
   24     0     0 ?        00:00:00 idle_inject/1
   25     0     0 ?        00:00:00 migration/1
   26     0     0 ?        00:00:03 ksoftirqd/1
   31     0     0 ?        00:00:00 kdevtmpfs
   32     0     0 ?        00:00:00 kworker/R-inet_frag_wq
   33     0     0 ?        00:00:00 rcu_tasks_kthread
   34     0     0 ?        00:00:00 rcu_tasks_rude_kthread
   35     0     0 ?        00:00:00 rcu_tasks_trace_kthread
   36     0     0 ?        00:00:00 kauditd
   37     0     0 ?        00:00:00 khungtaskd
   38     0     0 ?        00:00:00 oom_reaper
   39     0     0 ?        00:00:00 kworker/u10:1-events_unbound
   40     0     0 ?        00:00:00 kworker/u10:2-kvfree_rcu_reclaim
   41     0     0 ?        00:00:00 kworker/R-writeback
   42     0     0 ?        00:00:00 kcompactd0
   43     0     0 ?        00:00:00 ksmd
   44     0     0 ?        00:00:04 khugepaged
   45     0     0 ?        00:00:00 kworker/R-kblockd
   46     0     0 ?        00:00:00 kworker/R-blkcg_punt_bio
   47     0     0 ?        00:00:00 kworker/R-kintegrityd
   48     0     0 ?        00:00:00 irq/9-acpi
```

Fig. 1 -- Output di ps senza argomenti: visibilita' limitata a bash e ps (PID 1278, 1282)

Per ottenere visibilita' sull'intero sistema e' necessario elevare i privilegi con **sudo ps -e**. Il flag **-e** (equivalente a **-A**) estende la visualizzazione a tutti i processi di sistema.

Fig. 2 -- Output di sudo ps -e: PID 1 (systemd) fino ai processi kernel (kworker, kthreadd...)

```
[analyst@secOps ~]$ sudo ps -e
PID TTY          TIME CMD
 1 ?            00:00:01 systemd
 2 ?            00:00:00 kthreadd
 3 ?            00:00:00 pool_workqueue_release
 4 ?            00:00:00 kworker/R-rcu_gp
 5 ?            00:00:00 kworker/R-sync_wq
 6 ?            00:00:00 kworker/R-kvfree_rcu_reclaim
 7 ?            00:00:00 kworker/R-slub_flushwq
 8 ?            00:00:00 kworker/R-netns
12 ?            00:00:00 kworker/u8-0-events_unbound
13 ?            00:00:00 kworker/u8-1-ipv6_addrconf
14 ?            00:00:00 kworker/R-mm_percpu_wq
15 ?            00:00:00 ksoftirqd/0
16 ?            00:00:02 rcu_preempt
17 ?            00:00:00 rcub/0
18 ?            00:00:00 rcu_exp_par_gp_kthread_worker/0
19 ?            00:00:00 rcu_exp_gp_kthread_worker
20 ?            00:00:00 migration/0
21 ?            00:00:00 idle_inject/0
22 ?            00:00:00 cpuhp/0
23 ?            00:00:00 cpuhp/1
24 ?            00:00:00 idle_inject/1
25 ?            00:00:00 migration/1
26 ?            00:00:03 ksoftirqd/1
27 ?            00:00:00 kworker/1:0-events
31 ?            00:00:00 kdevtmpfs
32 ?            00:00:00 kworker/R-inet_frag_wq
33 ?            00:00:00 rcu_tasks_kthread
34 ?            00:00:00 rcu_tasks_rude_kthread
35 ?            00:00:00 rcu_tasks_trace_kthread
36 ?            00:00:00 kauditd
37 ?            00:00:00 khungtaskd
38 ?            00:00:00 oom_reaper
39 ?            00:00:00 kworker/u10:1-events_power_efficient
40 ?            00:00:00 kworker/u10:2-kvfree_rcu_reclaim
41 ?            00:00:00 kworker/R-writeback
42 ?            00:00:00 kcompactd0
43 ?            00:00:00 ksmd
44 ?            00:00:03 khugepaged
45 ?            00:00:00 kworker/R-kblockd
```

Fig. 2 -- Output di `sudo ps -e`: PID 1 (systemd) fino ai processi kernel**Concetto chiave**

In Linux i processi di sistema e i demoni vengono avviati dall'utente root e non sono visibili a un utente standard senza privilegi. L'escalation con **sudo** e' indispensabile per l'analisi forense di un host.

Fig. 3 -- Continuazione di `sudo ps -e`: processo `systemd-network` (PID 296), `sshd` (376), `vsftpd` (487)...

```
Terminal - analyst@secOps:~
File Edit View Terminal Tabs Help
[analyst@secOps ~]$ ps
  PID TTY          TIME CMD
1278 pts/1    00:00:00 bash
1282 pts/1    00:00:00 ps
[analyst@secOps ~]$
```

Fig. 3 -- Continuazione di `sudo ps -e`: `systemd-network`, `sshd`, `vsftpd`...Fig. 4 -- Segmento successivo: `ovsdb-server`, `python3.9`, `lightdm`, `Xorg`, `syslog-ng`, `xfce4`...

```

41 ? 00:00:00 kworker/R-writeback
42 ? 00:00:00 kcompactd0
43 ? 00:00:00 ksm
44 ? 00:00:00 khugepaged
45 ? 00:00:00 kworker/R-kblockd
46 ? 00:00:00 kworker/R-blkcg_punt_bio
47 ? 00:00:00 kworker/R-kintegrityd
48 ? 00:00:00 irq/9-acpi
49 ? 00:00:00 kworker/u9:1-events_unbound
50 ? 00:00:00 kworker/R-tpm_dev_wq
51 ? 00:00:00 kworker/R-acpi_sff
52 ? 00:00:00 kworker/R-edac-poller
53 ? 00:00:00 kworker/R-devfreq_wq
54 ? 00:00:00 watchdogd
55 ? 00:00:00 kworker/1:1H-kblockd
56 ? 00:00:00 kswapd0
57 ? 00:00:00 kworker/R-kthrotld
58 ? 00:00:00 kworker/R-acpi_thermal_pm
59 ? 00:00:00 scsi_eh_0
60 ? 00:00:00 kworker/R-scsi_tmf_0
61 ? 00:00:00 scsi_eh_1
62 ? 00:00:00 kworker/R-scsi_tmf_1
63 ? 00:00:00 kworker/R-mid
64 ? 00:00:00 kworker/0:1H-kblockd
65 ? 00:00:00 kworker/R-ipv6_addrconf
66 ? 00:00:00 kworker/R-kstrp
67 ? 00:00:00 kworker/R-zswap-shrink
68 ? 00:00:00 kworker/u11:0
69 ? 00:00:00 kworker/u13:0-ttm
70 ? 00:00:00 scsi_eh_2
71 ? 00:00:00 kworker/R-scsi_tmf_2
72 ? 00:00:00 scsi_eh_3
73 ? 00:00:00 kworker/R-scsi_tmf_3
74 ? 00:00:00 irq/18-vmwgfx
75 ? 00:00:00 kworker/0:2H-kblockd
76 ? 00:00:00 kworker/R-ttm
77 ? 00:00:00 jbd2/sda1-8
78 ? 00:00:00 kworker/R-ext4-rsv-conversion
79 ? 00:00:01 systemd-journal
80 ? 00:00:04 kworker/1:3-events
81 ? 00:00:00 kworker/R-iprt-VBoxQueue
82 ? 00:00:00 iprt-VBoxTscThread
83 ? 00:00:00 systemd-udev
84 ? 00:00:00 psimmon
85 ? 00:00:00 systemd-network

```

Fig. 4 -- Segmento successivo: ovsdb-server, python3.9, lightdm, Xorg, syslog-ng, xfce4...

1.2 -- Gerarchia dei Processi con ps -ejH

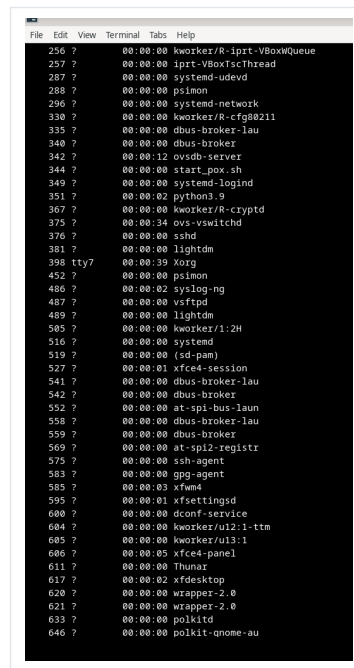
L'aggiunta dei flag **-jH** trasforma l'output in un albero gerarchico. Il flag **-j** include le colonne PGID e SID (process group e session ID), mentre **-H** indenta i processi figli verso destra, rendendo immediatamente visibili le relazioni padre-figlio.

Fig. 5 -- sudo ps -ejH: prima schermata con l'albero a partire da kthreadd (PID 2)

Nota

Nel documento originale questa schermata era presente come didascalia ma priva di immagine; lo screenshot corrispondente non è incluso nei materiali. Le figure successive coprono comunque l'intero albero.

Fig. 6 -- Continuazione dell'albero: processi kernel e demoni di sistema con PGID/SID



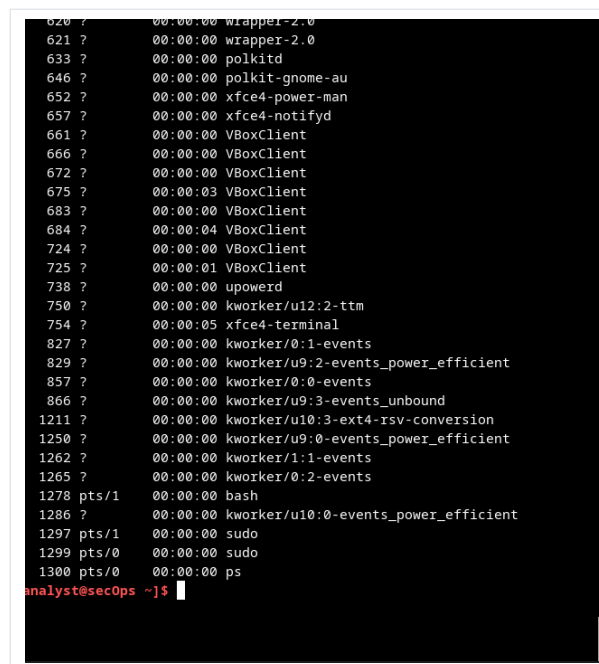
```

File Edit View Terminal Tabs Help
256 ? 00:00:00 kworker/R-irqt-VBoxQueue
257 ? 00:00:00 irqt-VBoxTscThread
287 ? 00:00:00 systemd-udev
288 ? 00:00:00 psimmon
296 ? 00:00:00 systemd-network
336 ? 00:00:00 kworker/R-cfg80211
335 ? 00:00:00 dbus-broker-lau
340 ? 00:00:00 dbus-broker
342 ? 00:00:12 ovsdb-server
344 ? 00:00:00 start_pox.sh
349 ? 00:00:00 systemd-logind
351 ? 00:00:00 python3.9
367 ? 00:00:00 kworker/R-cryptd
375 ? 00:00:34 ovs-vsctd
376 ? 00:00:00 sshd
381 ? 00:00:00 lightdm
398 tty7 00:00:39 xorg
452 ? 00:00:00 psimmon
486 ? 00:00:02 syslog-ng
487 ? 00:00:00 vsftpd
489 ? 00:00:00 lightdm
585 ? 00:00:00 kworker/1:2H
516 ? 00:00:00 systemd
519 ? 00:00:00 (sd-pam)
527 ? 00:00:01 xfce4-session
541 ? 00:00:00 dbus-broker-lau
542 ? 00:00:00 dbus-broker
552 ? 00:00:00 at-spi-bus-laun
558 ? 00:00:00 dbus-broker-lau
559 ? 00:00:00 dbus-broker
569 ? 00:00:00 at-spi2-registr
575 ? 00:00:00 ssh-agent
583 ? 00:00:00 gpg-agent
585 ? 00:00:03 xfwm4
595 ? 00:00:01 xfsettingsd
600 ? 00:00:00 dconf-service
604 ? 00:00:00 kworker/u12:1-ttm
605 ? 00:00:00 kworker/u13:1
606 ? 00:00:05 xfce4-panel
611 ? 00:00:00 Thunar
617 ? 00:00:02 xfdesktop
620 ? 00:00:00 wrapper-2.0
621 ? 00:00:00 wrapper-2.0
633 ? 00:00:00 polkitd
646 ? 00:00:00 polkit-gnome-au

```

Fig. 6 -- Continuation dell'albero: processi kernel e demoni di sistema con PGID/SID

Fig. 7 -- Gerarchia systemd (PID 1) -> systemd-journal, udevd, syslog-ng, sshd, lightdm...



```

620 ? 00:00:00 wrapper-2.0
621 ? 00:00:00 wrapper-2.0
633 ? 00:00:00 polkitd
646 ? 00:00:00 polkit-gnome-au
652 ? 00:00:00 xfce4-power-man
657 ? 00:00:00 xfce4-notifyd
661 ? 00:00:00 VBoxClient
666 ? 00:00:00 VBoxClient
672 ? 00:00:00 VBoxClient
675 ? 00:00:03 VBoxClient
683 ? 00:00:00 VBoxClient
684 ? 00:00:04 VBoxClient
724 ? 00:00:00 VBoxClient
725 ? 00:00:01 VBoxClient
738 ? 00:00:00 upowerd
750 ? 00:00:00 kworker/u12:2-ttm
754 ? 00:00:05 xfce4-terminal
827 ? 00:00:00 kworker/0:1-events
829 ? 00:00:00 kworker/u9:2-events_power_efficient
857 ? 00:00:00 kworker/0:0-events
866 ? 00:00:00 kworker/u9:3-events_unbound
1211 ? 00:00:00 kworker/u10:3-ext4-rsv-conversion
1250 ? 00:00:00 kworker/u9:0-events_power_efficient
1262 ? 00:00:00 kworker/1:1-events
1265 ? 00:00:00 kworker/0:2-events
1278 pts/1 00:00:00 bash
1286 ? 00:00:00 kworker/u10:0-events_power_efficient
1297 pts/1 00:00:00 sudo
1299 pts/0 00:00:00 sudo
1300 pts/0 00:00:00 ps
analyst@secOps ~]$

```

Fig. 7 -- Gerarchia systemd (PID 1) -> systemd-journal, udevd, syslog-ng, sshd, lightdm...

Fig. 8 -- Coda dell'albero: VBoxClient, xfce4-terminal, bash (1278) -> sudo (1324) -> ps (1327)

```

44 0 0 ? 00:00:04 khugepaged
45 0 0 ? 00:00:00 kworker/R-kblockd
46 0 0 ? 00:00:00 kworker/R-blkcg_punt_bio
47 0 0 ? 00:00:00 kworker/R-kintegrityd
48 0 0 ? 00:00:00 irq/9-acpi
49 0 0 ? 00:00:00 kworker/u9:1-events_power_efficient
50 0 0 ? 00:00:00 kworker/R-tpm_dev_wq
51 0 0 ? 00:00:00 kworker/R-ata_sff
52 0 0 ? 00:00:00 kworker/R-edac-poller
53 0 0 ? 00:00:00 kworker/R-devfreq_wq
54 0 0 ? 00:00:00 watchdogd
55 0 0 ? 00:00:00 kworker/1:1H-kblockd
56 0 0 ? 00:00:00 kswapd0
57 0 0 ? 00:00:00 kworker/R-kthrotld
58 0 0 ? 00:00:00 kworker/R-acpi_thermal_pm
59 0 0 ? 00:00:00 scsi_ah_0
60 0 0 ? 00:00:00 kworker/R-scsi_tmf_0
61 0 0 ? 00:00:00 scsi_ah_1
62 0 0 ? 00:00:00 kworker/R-scsi_tmf_1
63 0 0 ? 00:00:00 kworker/R-mld
64 0 0 ? 00:00:00 kworker/0:1H-kblockd
65 0 0 ? 00:00:00 kworker/R-ipv6_addrconf
66 0 0 ? 00:00:00 kworker/R-kstrp
67 0 0 ? 00:00:00 kworker/R-zswap-shrink
68 0 0 ? 00:00:00 kworker/u11:0
69 0 0 ? 00:00:00 kworker/u13:0-ttm
70 0 0 ? 00:00:00 scsi_ah_2
71 0 0 ? 00:00:00 kworker/R-scsi_tmf_2
72 0 0 ? 00:00:00 scsi_ah_3
73 0 0 ? 00:00:00 kworker/R-scsi_tmf_3
74 0 0 ? 00:00:00 irq/18-vmwgfx
75 0 0 ? 00:00:00 kworker/0:2H-kblockd
76 0 0 ? 00:00:00 kworker/R-ttm
77 0 0 ? 00:00:00 jbd2/sda1-8
78 0 0 ? 00:00:00 kworker/R-ext4_rsv-conversion
79 0 0 ? 00:00:04 kworker/1:3-mm_percpu_wq
80 0 0 ? 00:00:00 kworker/R-iprt-VBoxQueue
81 0 0 ? 00:00:00 iprt-VBoxTscThread
82 0 0 ? 00:00:00 psimon
83 0 0 ? 00:00:00 kworker/R-cfg80211
84 0 0 ? 00:00:00 kworker/R-cryptd
85 0 0 ? 00:00:00 psimon
86 0 0 ? 00:00:00 kworker/1:2H
87 0 0 ? 00:00:00 kworker/u12:1-ttm
88 0 0 ? 00:00:00 kworker/u13:1

```

Fig. 8 -- Coda dell'albero: VBoxClient, xfce4-terminal, bash -> sudo -> ps

Osservazione didattica

Dall'albero emerge che **bash** (PID 1278) ha avviato **sudo** (1324), il quale a sua volta ha lanciato **ps** (1327). Questa catena padre-figlio e' la prova visiva del meccanismo di **fork/exec** di Linux.

Attenzione

Il processo **grep** compare anch'esso nell'output quando si filtra per PID, perche' viene catturato "in volo" mentre e' in esecuzione al momento dello snapshot.

- 1.3 -- Mappatura delle Porte con netstat -tunap**

Il comando **sudo netstat -tunap** restituisce la mappa completa dei socket di rete. Ogni flag contribuisce in modo preciso:

Flag	Significato
-t	Mostra connessioni TCP
-u	Mostra connessioni UDP
-n	Risoluzione numerica (no DNS reverse lookup)
-a	Tutti i socket, inclusi quelli in LISTEN
-p	Mostra PID e nome del programma associato

La prima esecuzione di **netstat** ha rivelato un'assenza inattesa: **nginx** non compariva tra le porte in ascolto. Il demone non era stato avviato. Questo e' un dato operativo reale, non un errore: i servizi su Linux non si avviano automaticamente se non configurati.

Fig. 9 -- Primo netstat: nginx assente. Visibili python3.9 (6633), vsftpd (21), sshd (22), systemd-network UDP (68)


```

750 0 0 ? 00:00:00 kworker/u12:2-ttm
827 0 0 ? 00:00:00 kworker/0:1-events
866 0 0 ? 00:00:00 kworker/u9:3-events_unbound
1211 0 0 ? 00:00:00 kworker/u10:3-kvfree_rcu_reclaim
1250 0 0 ? 00:00:00 kworker/u9:0-events_power_efficient
1262 0 0 ? 00:00:00 kworker/1:1-events
1265 0 0 ? 00:00:00 kworker/0:2-events
1286 0 0 ? 00:00:00 kworker/u10:0-events_unbound
1321 0 0 ? 00:00:00 kworker/u9:2-events_power_efficient
1 1 1 ? 00:00:01 systemd
241 241 241 ? 00:00:01 systemd-journal
287 287 287 ? 00:00:00 systemd-udev
296 296 296 ? 00:00:00 systemd-network
335 335 335 ? 00:00:00 dbus-broker-lau
340 335 335 ? 00:00:00 dbus-broker
342 342 342 ? 00:00:12 ovsdb-server
344 344 344 ? 00:00:00 start_pox.sh
351 344 344 ? 00:00:02 python3.9
349 349 349 ? 00:00:00 systemd-logind
375 375 375 ? 00:00:36 ovs-vswhd
376 376 376 ? 00:00:00 sshd
381 381 381 ? 00:00:00 lightdm
398 398 398 tty? 00:00:43 xorg
489 381 381 ? 00:00:00 lightdm
527 527 527 ? 00:00:01 xfce4-session
585 527 527 ? 00:00:04 xfwm4
595 527 527 ? 00:00:01 xfsettingsd
606 527 527 ? 00:00:05 xfce4-panel
620 527 527 ? 00:00:00 wrapper-2.0
621 527 527 ? 00:00:00 wrapper-2.0
611 527 527 ? 00:00:00 Thunar
617 527 527 ? 00:00:02 xfdesktop
646 527 527 ? 00:00:00 polkit-gnome-au
652 527 527 ? 00:00:00 xfce4-power-man
486 486 486 ? 00:00:02 syslog-ng
487 487 487 ? 00:00:00 vsftpd
516 516 516 ? 00:00:00 systemd
519 516 516 ? 00:00:00 (sd-pam)
541 541 541 ? 00:00:00 dbus-broker-lau
542 541 541 ? 00:00:00 dbus-broker
552 552 552 ? 00:00:00 at-spi-bus-lau
558 552 552 ? 00:00:00 dbus-broker-lau
559 552 552 ? 00:00:00 dbus-broker
569 569 569 ? 00:00:00 at-spi2-registr
583 583 583 ? 00:00:00 qpa-agent

```

Fig. 9 -- Primo netstat: nginx assente. Visibili python3.9, vsftpd, sshd, systemd-network

Dopo l'avvio manuale di nginx, la seconda esecuzione ha confermato la presenza della porta 80:

Fig. 10 -- Secondo netstat: nginx master process (PID 1386) ora in ascolto su 0.0.0.0:80 TCP -- LISTEN

```

600 600 600 ? 00:00:00 dconf-service
657 657 657 ? 00:00:00 xfce4-notifyd
575 575 575 ? 00:00:00 ssh-agent
633 633 633 ? 00:00:00 polkitd
661 660 660 ? 00:00:00 VBoxClient
666 660 660 ? 00:00:00 VBoxClient
672 670 670 ? 00:00:00 VBoxClient
675 670 670 ? 00:00:03 VBoxClient
683 682 682 ? 00:00:00 VBoxClient
684 682 682 ? 00:00:05 VBoxClient
724 723 723 ? 00:00:00 VBoxClient
725 723 723 ? 00:00:01 VBoxClient
738 738 738 ? 00:00:00 upowerd
754 527 527 ? 00:00:06 xfce4-terminal
1278 1278 1278 pts/1 00:00:00 bash
1324 1324 1278 pts/1 00:00:00 sudo
1326 1326 1326 pts/0 00:00:00 sudo
1327 1327 1326 pts/0 00:00:00 ps
[analyst@secOps ~]$

```

Fig. 10 -- Secondo netstat: nginx master process (PID 1386) in ascolto su 0.0.0.0:80

Porta	Protocollo	Stato	PID / Processo	Servizio
80	TCP	LISTEN	1386 / nginx	Web server HTTP
21	TCP	LISTEN	487 / vsftpd	FTP server
22	TCP	LISTEN	376 / sshd	Secure Shell

Porta	Protocollo	Stato	PID / Processo	Servizio
6633	TCP	LISTEN	351 / python3.9	Servizio custom (OpenFlow?)
68	UDP	--	296 / systemd-network	Client DHCP

• 1.4 -- Analisi dell'Architettura di Sicurezza di Nginx

Filtrando i processi attivi con **ps -ef | grep 1386** emerge la struttura interna di nginx, che implementa un modello master/worker a doppio processo per motivi di sicurezza:

Fig. 11 -- ps -ef | grep 1386: master process (root, PID 1386) e worker process (http, PID 1387)

```
[analyst@secOps ~]$ sudo netstat -tunap
[sudo] password for analyst:
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:6633            0.0.0.0:*               LISTEN      351/python3.9
tcp        0      0 0.0.0.0:21              0.0.0.0:*               LISTEN      487/vsftpd
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      376/sshd: /usr/bin/
tcp6       0      0 :::22                   :::*                    LISTEN      376/sshd: /usr/bin/
udp        0      0 0.0.0.0:68              0.0.0.0:*               LISTEN      296/systemd-network
[analyst@secOps ~]$
```

Fig. 11 -- ps -ef | grep 1386: master process (root) e worker process (http)

PID	Utente	Ruolo	Implicazione di sicurezza
1386	root	Master Process	Gestisce configurazione, binding porte privilegiate (<1024), segnali
1387	http	Worker Process	Gestisce le richieste in ingresso con privilegi minimi
1403	analyst	grep 1386	Processo grep, visibile perche' attivo durante lo snapshot

Principio di sicurezza -- Privilege Separation

Il worker process gira come utente "http" (non-privilegiato). Se un attaccante sfruttasse una vulnerabilita' in nginx, otterrebbe solo i privilegi dell'utente http, non quelli di root. Questo pattern e' adottato anche da Apache, OpenSSH e altri demoni critici.

03

Parte 2 -- Validazione dei Servizi TCP

La sola lettura di **ps** e **netstat** fornisce informazioni basate su metadati di processo: il nome del programma dichiarato dal sistema. Il **banner grabbing** consente di interagire direttamente col protocollo, ottenendo la firma autentica del servizio remoto indipendentemente dal nome del processo. Questa tecnica e' fondamentale durante un penetration test o un'analisi delle vulnerabilita'.

Telnet trasmette tutto in chiaro

Credenziali, comandi e risposte viaggiano senza cifratura. Telnet non va mai usato per l'amministrazione remota in produzione: il suo utilizzo e' legittimo solo come strumento diagnostico in ambienti controllati.

- 2.1 -- Test del Web Server -- Porta 80 (HTTP)

Connessione: **telnet 127.0.0.1 80**

Fig. 12 -- Telnet porta 80: il server risponde con HTTP/1.1 400 Bad Request e rivela "Server: nginx/1.28.0"

```
[analyst@secOps ~]$ sudo netstat -tunap
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:6633           0.0.0.0:*               LISTEN      351/python3.9
tcp        0      0 0.0.0.0:80             0.0.0.0:*               LISTEN      1386/nginx: master
tcp        0      0 0.0.0.0:21             0.0.0.0:*               LISTEN      487/vsftpd
tcp        0      0 0.0.0.0:22             0.0.0.0:*               LISTEN      376/sshd: /usr/bin/
tcp6       0      0 :::22                  :::*                    LISTEN      376/sshd: /usr/bin/
udp        0      0 10.0.2.15:68           0.0.0.0:*               *          296/systemd-network
```

Fig. 12 -- Telnet porta 80: risposta HTTP/1.1 400 Bad Request, Server: nginx/1.28.0

Alla connessione TCP e' stato inviato del testo casuale ("ciao"). Il server ha risposto con una pagina HTTP 400 Bad Request, tipica risposta quando il client non rispetta la sintassi HTTP/1.1. Il dato piu' rilevante emerge dall'header **Server**:

Discrepanza documentale rilevata

La documentazione teorica del laboratorio indicava la versione **nginx/1.16.1**, ma il banner grabbing ha restituito **nginx/1.28.0**. Questo dimostra che la documentazione era obsoleta. In un assessment reale, questa discrepanza andrebbe registrata come *finding*: versione non documentata in esecuzione.

Campo	Valore rilevato
Protocollo risposta	HTTP/1.1
Codice di stato	400 Bad Request (input non valido)
Server banner	nginx/1.28.0
Content-Type	text/html
Comportamento	Connessione chiusa dal server dopo la risposta

- 2.2 -- Test del Server SSH -- Porta 22

Connessione: **telnet 127.0.0.1 22**

Fig. 13 -- Telnet porta 22: banner SSH-2.0-OpenSSH_10.0, poi disconnessione immediata per stringa non valida

```
[analyst@secOps ~]$ ps -ef | grep 1386
root      1386      1  0 07:24 ?        00:00:00 nginx: master process /usr/bin/nginx
http      1387     1386  0 07:24 ?        00:00:00 nginx: worker process
analyst    1403     1278  0 07:27 pts/1    00:00:00 grep 1386
```

Fig. 13 -- Telnet porta 22: banner SSH-2.0-OpenSSH_10.0, disconnessione immediata

A differenza di HTTP, il protocollo SSH espone il proprio banner immediatamente alla connessione TCP, prima ancora di ricevere dati dal client. Questo e' il comportamento standard: il server "presenta" la propria versione per negoziare il protocollo.

Inviando una stringa di testo non formattata, il demone ha rilevato l'assenza dell'handshake crittografico previsto e ha terminato la sessione con il messaggio "Invalid SSH identification string".

Campo	Valore rilevato
Banner	SSH-2.0-OpenSSH_10.0
Versione protocollo	SSH-2.0 (SSH versione 2)
Implementazione	OpenSSH 10.0
Comportamento anomalo	Disconnessione immediata se l'handshake non viene avviato

Nota di sicurezza

Il banner SSH e' esposto prima dell'autenticazione: e' intenzionale e necessario per la negoziazione del protocollo. Tuttavia rivela la versione esatta del software, utile per un attaccante che voglia cercare CVE specifici. Alcune hardening guide raccomandano di oscurare il banner o di omettere la versione minore.

● 2.3 -- Test della Porta UDP 68 -- Fallimento Atteso

Tentativo di connessione: **telnet 127.0.0.1 68**

Fig. 14 -- Telnet porta 68: "Unable to connect to remote host: Connection refused" -- risultato atteso

```
[analyst@secOps ~]$ telnet 127.0.0.1 80
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
ciao
HTTP/1.1 400 Bad Request
Server: nginx/1.28.0
Date: Mon, 22 Jun 2026 11:29:33 GMT
Content-Type: text/html
Content-Length: 157
Connection: close

<html>
<head><title>400 Bad Request</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<hr><center>nginx/1.28.0</center>
</body>
</html>
Connection closed by foreign host.
```

Fig. 14 -- Telnet porta 68: Connection refused -- risultato atteso

Il tentativo di connessione alla porta 68 ha prodotto un errore "Connection refused". Questo e' il risultato corretto e atteso, non un malfunzionamento.

Aspetto	Spiegazione
Porta 68	Usata da systemd-network come client DHCP (non come server)
Protocollo	UDP -- il DHCP opera esclusivamente su UDP (68 client, 67 server)
Perche' fallisce	Telnet crea connessioni TCP. TCP e UDP sono protocolli di Livello 4 incompatibili
Connection refused	Il kernel Linux rifiuta attivamente le connessioni TCP a porte che non hanno socket TCP in ascolto

Approfondimento

"Connection refused" in TCP significa che il kernel ha risposto con un pacchetto **RST** (reset). Questo e' diverso da "Connection timed out" (nessuna risposta = firewall drop). La distinzione e' importante durante il port scanning: RST = porta chiusa ma host raggiungibile; timeout = porta filtrata da firewall.

04

Conclusioni e Sintesi Comparativa

Le attività svolte hanno permesso di costruire un quadro completo dei servizi attivi sull'host, integrando due livelli di analisi complementari: l'ispezione interna dei processi e la validazione esterna tramite interazione col protocollo.

Strumento	Tecnica	Informazioni ottenute	Limitazione
ps	Analisi processi	PID, utente, gerarchia padre-figlio	Nomi dichiarati dal processo, non verificati
netstat	Mappatura socket	Porta, protocollo, stato, PID	Non verifica l'identità reale del servizio
telnet	Banner grabbing	Versione reale del software, comportamento protocollo	Solo TCP, trasmissione in chiaro

La discrepanza rilevata tra la versione nginx attesa (1.16.1) e quella reale (1.28.0) è l'esempio concreto del motivo per cui il banner grabbing è parte integrante di ogni analisi delle vulnerabilità: la documentazione invecchia, il software viene aggiornato, e solo l'interazione diretta col servizio rivela lo stato reale del sistema.

Raccomandazione finale

In ambienti di produzione, il banner grabbing non autorizzato verso sistemi di terzi è illegale. Queste tecniche vanno applicate esclusivamente su sistemi per cui si ha esplicita autorizzazione scritta.

Documento redatto a scopo didattico -- Cyber Security & Ethical Hacking, EPICODE. Attività su VM CyberOps in ambiente controllato.